Battery Intelligence Engine Using ESP32

Abstract

The Battery Intelligence Engine is an ESP32-based battery monitoring and protection platform designed to improve battery safety, reliability, and operational awareness. The system continuously monitors four battery cells, analyzes battery health, detects abnormal operating conditions, and performs automatic protection actions when required.

The project implements an event-driven architecture using non-blocking timing techniques based on `millis()`, ensuring responsive operation without the use of `delay()`. Safety mechanisms include overvoltage protection, undervoltage protection, sensor anomaly detection, cell imbalance analysis, relay-based battery isolation, buzzer alarms, LCD notifications, and cloud monitoring through Blynk.

---

1. Introduction

Battery management and protection systems are critical components in modern electric vehicles, energy storage systems, and portable electronic devices. Improper battery operation can lead to reduced performance, permanent damage, or safety hazards.

This project presents a Battery Intelligence Engine capable of continuously monitoring battery parameters and taking corrective actions when abnormal conditions occur. The ESP32 microcontroller serves as the central processing unit and integrates local monitoring, fault diagnostics, safety protection, and cloud connectivity into a single platform.

---

2. Objectives

The primary objectives of this project are:

- Monitor four battery cell voltages in real time.
- Detect overvoltage and undervoltage conditions.
- Identify sensor anomalies and invalid readings.
- Calculate battery health indicators.
- Detect cell imbalance conditions.
- Provide relay-based battery isolation.
- Generate audible and visual fault alerts.
- Display battery information on an LCD.
- Enable remote monitoring through Blynk IoT.
- Implement a fully non-blocking event-driven architecture.

---

3. Hardware Components

Component	Quantity
ESP32 DevKit V1	1
Potentiometer (Cell Simulation)	4
Relay Module	1
Buzzer	1
LED Indicator	1
16x2 I2C LCD	1
Connecting Wires	As Required

---

4. Circuit Description

The system uses four potentiometers to simulate battery cell voltages. Each potentiometer output is connected to an ESP32 analog input pin.

The ESP32 performs voltage measurements and battery analytics. When a fault condition is detected, the relay disconnects the battery pack while the buzzer and LED provide immediate fault indication. The LCD displays battery status and warning messages. Blynk cloud services enable remote monitoring and control.

---

5. System Architecture

```
Cell 1 Sensor
      |
Cell 2 Sensor
      |
Cell 3 Sensor
      |
Cell 4 Sensor
      |
      v
+----------------------+
|        ESP32         |
| Battery Intelligence |
|        Engine        |
+----------------------+
```

```
      |      |      |
      |      |      |
      v      v      v
    Relay Buzzer LCD
      |
      v
Battery Isolation

      |
      v
 Blynk Cloud
```

---

6. Working Principle

Step 1: Data Acquisition

The ESP32 continuously reads analog voltage values from all four battery cells.

Step 2: Battery Analytics

The Battery Intelligence Engine calculates:

- Maximum Cell Voltage
- Minimum Cell Voltage
- Average Cell Voltage
- Voltage Spread
- Cell Imbalance Percentage

The imbalance percentage is calculated using:

```
Imbalance (%) =
((Maximum Voltage - Minimum Voltage)
 / Average Voltage) × 100
```

Step 3: Fault Detection

The system checks for:

Overvoltage

```
Cell Voltage > 4.25 V
```

Undervoltage

```
Cell Voltage < 3.00 V
```

Sensor Anomaly

```
Invalid Voltage < 0.1 V
or
Invalid Voltage > 4.49 V
```

Cell Imbalance

```
Excessive voltage difference
between cells
```

Step 4: Safety Protection

When a critical fault is detected:

- Relay disconnects the battery.
- Buzzer alarm is activated.
- LCD displays warning information.
- Cloud dashboard receives fault updates.

Step 5: Recovery

Once the fault condition is cleared and remains stable for the configured recovery period, the system returns to normal operation.

---

7. Event-Driven Software Architecture

The project follows a fully non-blocking event-driven architecture.

No `delay()` functions are used.

Instead, all operations are scheduled using:

```
millis()
```

The software consists of:

- Battery Intelligence Engine
- Fault Tolerant Kernel
- Safety Protection Kernel
- Human Machine Interface (HMI)
- Cloud Telemetry Manager

This design improves responsiveness and reliability.

---

8. Runtime States

The system operates in four major runtime states.

NORMAL

- Battery operating normally.
- Relay remains connected.
- LCD displays battery analytics.

DEGRADED

- Sensor anomaly detected.
- System continues operation with warnings.

FAILSAFE

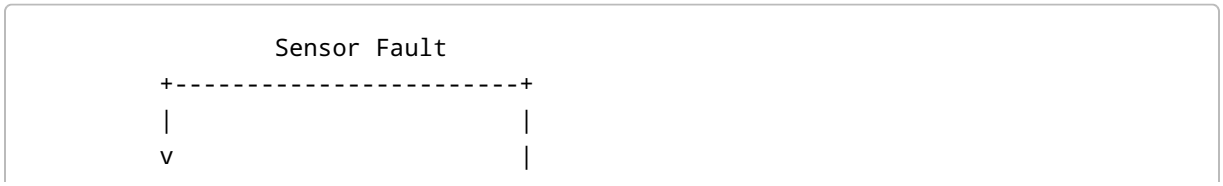
- Critical battery fault detected.
- Relay disconnects battery.
- Audible alarm activated.

SHUTDOWN

- Remote shutdown command received.
- Battery isolated from load.

---

9. State Machine



```
+-----------------+        +-----------------+
|     NORMAL      | -----> |    DEGRADED     |
+-----------------+        +-----------------+
         |
         |
         | OV / UV / Imbalance
         v
+-----------------+
|    FAILSAFE     |
+-----------------+
         |
         |
         | Remote Shutdown
         v
+-----------------+
|    SHUTDOWN     |
+-----------------+

Fault Cleared
       |
       +---------> NORMAL
```

---

10. Features Implemented

- Real-time battery monitoring
- Multi-cell voltage acquisition
- Battery analytics engine
- Overvoltage protection
- Undervoltage protection
- Cell imbalance detection
- Sensor anomaly detection
- Relay-based protection
- Buzzer warning system
- LCD dashboard
- Blynk IoT integration
- Event-driven architecture
- State machine implementation
- Recovery logic
- Non-blocking execution

---

11. Results

The Battery Intelligence Engine successfully monitored all four battery cells and detected abnormal operating conditions. Faults were identified in real time, and protective actions were executed immediately. The LCD provided local diagnostics while Blynk enabled remote monitoring.

Testing demonstrated reliable operation of the safety protection mechanisms and event-driven architecture.

---

12. Conclusion

The Battery Intelligence Engine demonstrates an effective implementation of a modern battery monitoring and protection system using ESP32. The project combines real-time analytics, safety protection, local visualization, and cloud connectivity within a scalable event-driven architecture.

The system successfully improves battery safety and provides a strong foundation for future automotive and energy storage applications.

Future Enhancements

- Current sensing integration
- Temperature monitoring
- State of Charge (SOC) estimation
- State of Health (SOH) estimation
- Data logging to SD card
- AI-based fault prediction
- CAN Bus communication
- Automotive-grade battery balancing

Author

Surya Pratap

B.Tech Electronics and Communication Engineering

ESP32 Battery Intelligence Engine Project